

Объектно-ориентированное программирование

Лекция 11: Структурные паттерны (Часть 1)

Adapter (Адаптер). Facade (Фасад). Bridge (Мост). Proxy (Заместитель).

В структурных паттернах рассматривается вопрос о том, как из классов и объектов образуются более крупные структуры.

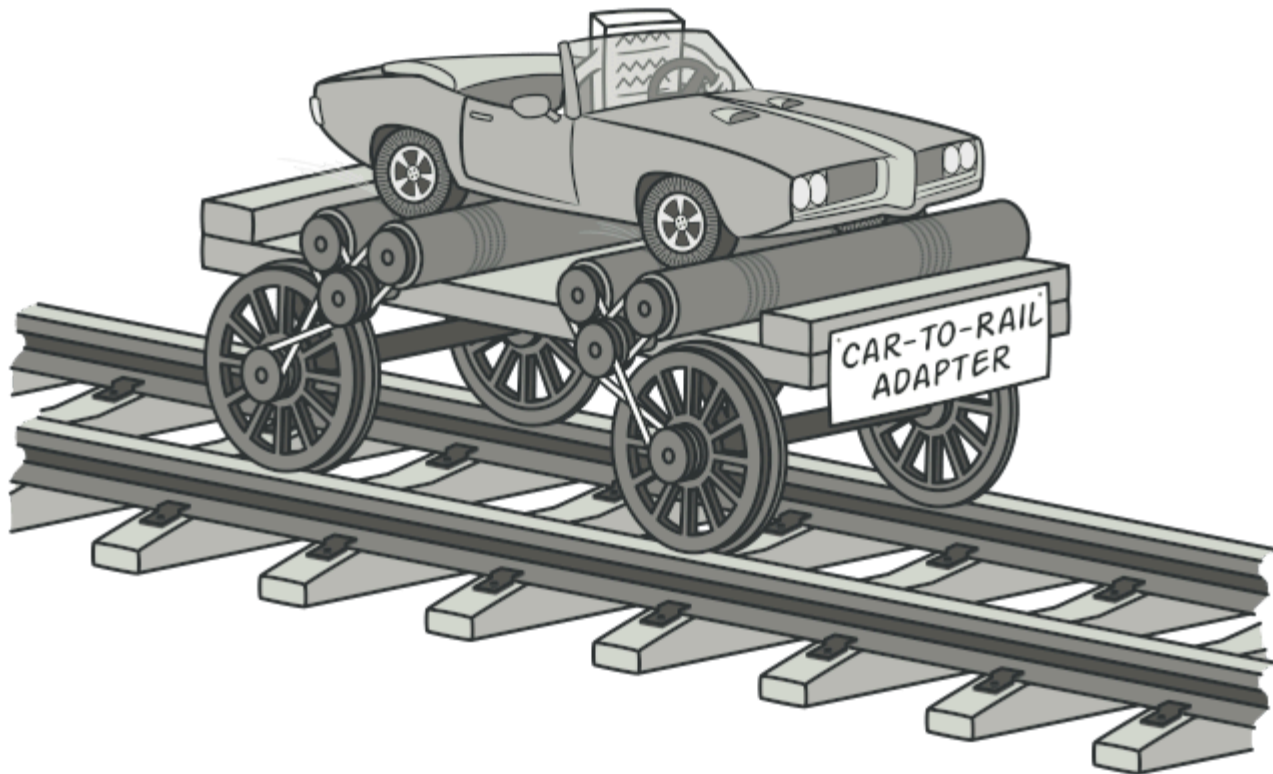
Структурные паттерны уровня класса используют наследование для составления композиций из интерфейсов и реализаций.

Вместо композиции интерфейсов или реализаций структурные паттерны уровня объекта komponуют объекты для получения новой функциональности. Дополнительная гибкость в этом случае связана с возможностью изменить композицию объектов во время выполнения, что недопустимо для статической композиции классов.



Университет
Сириус
Колледж

Паттерн Адаптер (Adapter)



Паттерн Адаптер (Adapter)

Преобразует интерфейс одного класса в интерфейс другого, который ожидают клиенты. Адаптер обеспечивает совместную работу классов с несовместимыми интерфейсами, которая без него была бы невозможна.

Используется, когда вы:



Паттерн Адаптер (Adapter)

Преобразует интерфейс одного класса в интерфейс другого, который ожидают клиенты. Адаптер обеспечивает совместную работу классов с несовместимыми интерфейсами, которая без него была бы невозможна.

Используется, когда вы:

- хотите использовать существующий класс, но его интерфейс не соответствует вашим потребностям;

Преобразует интерфейс одного класса в интерфейс другого, который ожидают клиенты. Адаптер обеспечивает совместную работу классов с несовместимыми интерфейсами, которая без него была бы невозможна.

Используется, когда вы:

- хотите использовать существующий класс, но его интерфейс не соответствует вашим потребностям;
- собираетесь создать повторно используемый класс, который должен взаимодействовать с заранее неизвестными или не связанными с ним классами, имеющими несовместимые интерфейсы;

Паттерн Адаптер (Adapter)

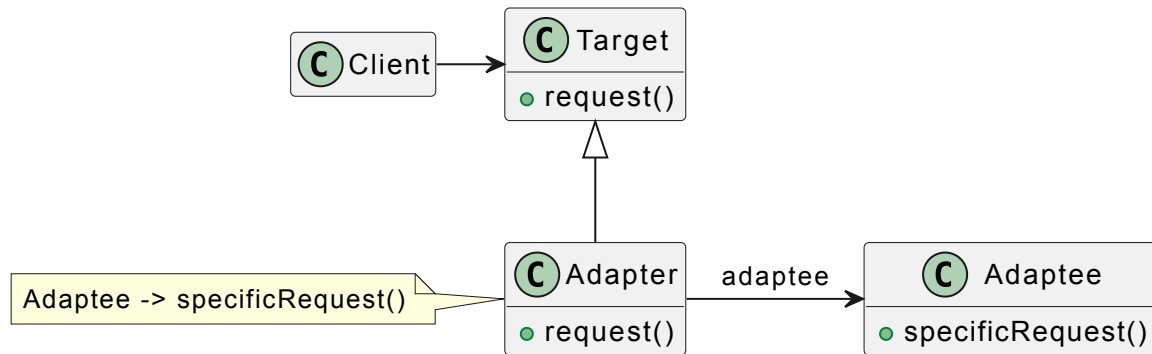
Преобразует интерфейс одного класса в интерфейс другого, который ожидают клиенты. Адаптер обеспечивает совместную работу классов с несовместимыми интерфейсами, которая без него была бы невозможна.

Используется, когда вы:

- хотите использовать существующий класс, но его интерфейс не соответствует вашим потребностям;
- собираетесь создать повторно используемый класс, который должен взаимодействовать с заранее неизвестными или не связанными с ним классами, имеющими несовместимые интерфейсы;
- (только для адаптера объектов!) нужно использовать несколько существующих подклассов, но непрактично адаптировать их интерфейсы путем порождения новых подклассов от каждого. В этом случае адаптер объектов может приспособливать интерфейс их общего родительского класса.



Паттерн Адаптер (Adapter)





Паттерн Адаптер (Adapter)

```
1 class OldSystem:
2     def get_data_xml(self):
3         return "<data>123</data>"
4
5 class NewSystem:
6     def process_json(self, json_data):
7         print(f"Processing: {json_data}")
8
9 class Adapter:
10     def __init__(self, old_system):
11         self.old_system = old_system
12
13     def get_data_json(self):
14         xml = self.old_system.get_data_xml()
15         # Впрошенная логика конвертации
```



Паттерн Адаптер (Adapter)

```
1 class OldSystem:
2     def get_data_xml(self):
3         return "<data>123</data>"
4
5 class NewSystem:
6     def process_json(self, json_data):
7         print(f"Processing: {json_data}")
8
9 class Adapter:
10     def __init__(self, old_system):
11         self.old_system = old_system
12
13     def get_data_json(self):
14         xml = self.old_system.get_data_xml()
15         # Впрошенная логика конвертации
```



Паттерн Адаптер (Adapter)

```
1 class OldSystem:
2     def get_data_xml(self):
3         return "<data>123</data>"
4
5 class NewSystem:
6     def process_json(self, json_data):
7         print(f"Processing: {json_data}")
8
9 class Adapter:
10     def __init__(self, old_system):
11         self.old_system = old_system
12
13     def get_data_json(self):
14         xml = self.old_system.get_data_xml()
15         # Встроенная логика конвертации
```



Паттерн Адаптер (Adapter)

```
1 class OldSystem:
2     def get_data_xml(self):
3         return "<data>123</data>"
4
5 class NewSystem:
6     def process_json(self, json_data):
7         print(f"Processing: {json_data}")
8
9 class Adapter:
10     def __init__(self, old_system):
11         self.old_system = old_system
12
13     def get_data_json(self):
14         xml = self.old_system.get_data_xml()
15         # Впрошенная логика конвертации
```



Паттерн Адаптер (Adapter)

```
1 class OldSystem:
2     def get_data_xml(self):
3         return "<data>123</data>"
4
5 class NewSystem:
6     def process_json(self, json_data):
7         print(f"Processing: {json_data}")
8
9 class Adapter:
10     def __init__(self, old_system):
11         self.old_system = old_system
12
13     def get_data_json(self):
14         xml = self.old_system.get_data_xml()
15         # Встроенная логика конвертации
```



Паттерн Адаптер (Adapter)

```
6     def process_json(self, json_data):
7         print(f"Processing: {json_data}")
8
9     class Adapter:
10         def __init__(self, old_system):
11             self.old_system = old_system
12
13         def get_data_json(self):
14             xml = self.old_system.get_data_xml()
15             # Упрощенная логика конвертации
16             return {"data": xml.replace("<data>", "").replace("</data>", "")}
17
18 # Клиент работает с новым форматом, но получает данные из старого
19 old = OldSystem()
20 adapter = Adapter(old)
```



Паттерн Адаптер (Adapter)

```
4
5 class NewSystem:
6     def process_json(self, json_data):
7         print(f"Processing: {json_data}")
8
9 class Adapter:
10     def __init__(self, old_system):
11         self.old_system = old_system
12
13     def get_data_json(self):
14         xml = self.old_system.get_data_xml()
15         # Упрощенная логика конвертации
16         return {"data": xml.replace("<data>", "").replace("</data>", "")}
17
18 # Клиент работает с новым форматом, но получает данные из старого
```



Паттерн Адаптер (Adapter)

```
7         print('Processing: {json_data} ')
8
9     class Adapter:
10         def __init__(self, old_system):
11             self.old_system = old_system
12
13         def get_data_json(self):
14             xml = self.old_system.get_data_xml()
15             # Упрощенная логика конвертации
16             return {"data": xml.replace("<data>", "").replace("</data>", "")}
17
18 # Клиент работает с новым форматом, но получает данные из старого
19 old = OldSystem()
20 adapter = Adapter(old)
21 NewSystem().process_json(adapter.get_data_json())
```




Паттерн Адаптер (Adapter)

```
7         print(f"Processing: {json_data}")
8
9     class Adapter:
10         def __init__(self, old_system):
11             self.old_system = old_system
12
13         def get_data_json(self):
14             xml = self.old_system.get_data_xml()
15             # Упрощенная логика конвертации
16             return {"data": xml.replace("<data>", "").replace("</data>", "")}
17
18 # Клиент работает с новым форматом, но получает данные из старого
19 old = OldSystem()
20 adapter = Adapter(old)
21 NewSystem().process_json(adapter.get_data_json())
```



Паттерн Адаптер (Adapter)

```
7         print( f 'Processing: {json_data}' )
8
9     class Adapter:
10         def __init__(self, old_system):
11             self.old_system = old_system
12
13         def get_data_json(self):
14             xml = self.old_system.get_data_xml()
15             # Упрощенная логика конвертации
16             return {"data": xml.replace("<data>", "").replace("</data>", "")}
17
18 # Клиент работает с новым форматом, но получает данные из старого
19 old = OldSystem()
20 adapter = Adapter(old)
21 NewSystem().process_json(adapter.get_data_json())
```



Паттерн Адаптер (Adapter)

```
7         print(' Processing: {json_data} ')
8
9     class Adapter:
10         def __init__(self, old_system):
11             self.old_system = old_system
12
13         def get_data_json(self):
14             xml = self.old_system.get_data_xml()
15             # Упрощенная логика конвертации
16             return {"data": xml.replace("<data>", "").replace("</data>", "")}
17
18 # Клиент работает с новым форматом, но получает данные из старого
19 old = OldSystem()
20 adapter = Adapter(old)
21 NewSystem().process_json(adapter.get_data_json())
```

Адаптер класса:

Адаптер класса:

Адаптер класса:

- адаптирует Adaptee к Target , перепоручая действия конкретному классу Adaptee .
Поэтому данный паттерн не будет работать, если мы захотим одновременно адаптировать класс и его подклассы;

Адаптер класса:

- адаптирует Adaptee к Target, перепоручая действия конкретному классу Adaptee. Поэтому данный паттерн не будет работать, если мы захотим одновременно адаптировать класс и его подклассы;

Адаптер класса:

- адаптирует Adaptee к Target, перепоручая действия конкретному классу Adaptee. Поэтому данный паттерн не будет работать, если мы захотим одновременно адаптировать класс и его подклассы;
- позволяет адаптеру Adapter заместить некоторые операции адаптируемого класса Adaptee так как Adapter есть не что иное, как подкласс Adaptee ;

Адаптер класса:

- адаптирует Adaptee к Target, перепоручая действия конкретному классу Adaptee. Поэтому данный паттерн не будет работать, если мы захотим одновременно адаптировать класс и его подклассы;
- позволяет адаптеру Adapter заместить некоторые операции адаптируемого класса Adaptee так как Adapter есть не что иное, как подкласс Adaptee ;

Адаптер класса:

- адаптирует Adaptee к Target, перепоручая действия конкретному классу Adaptee. Поэтому данный паттерн не будет работать, если мы захотим одновременно адаптировать класс и его подклассы;
- позволяет адаптеру Adapter заместить некоторые операции адаптируемого класса Adaptee, так как Adapter есть не что иное, как подкласс Adaptee;
- вводит только один новый объект. Чтобы добраться до адаптируемого класса, не нужно никакого дополнительного обращения по указателю.

Адаптер объектов:

Адаптер объектов:

Адаптер объектов:

- позволяет одному адаптеру Adapter **работать со многим адаптируемыми объектами** **Adaptee** то есть с самим Adaptee и его подклассами (если таковые имеются). Адаптер может добавить новую функциональность сразу всем адаптируемым объектам;

Адаптер объектов:

- позволяет одному адаптеру Adapter **работать со многим адаптируемыми объектами** **Adaptee** то есть с самим Adaptee и его подклассами (если таковые имеются). Адаптер может добавить новую функциональность сразу всем адаптируемым объектам;

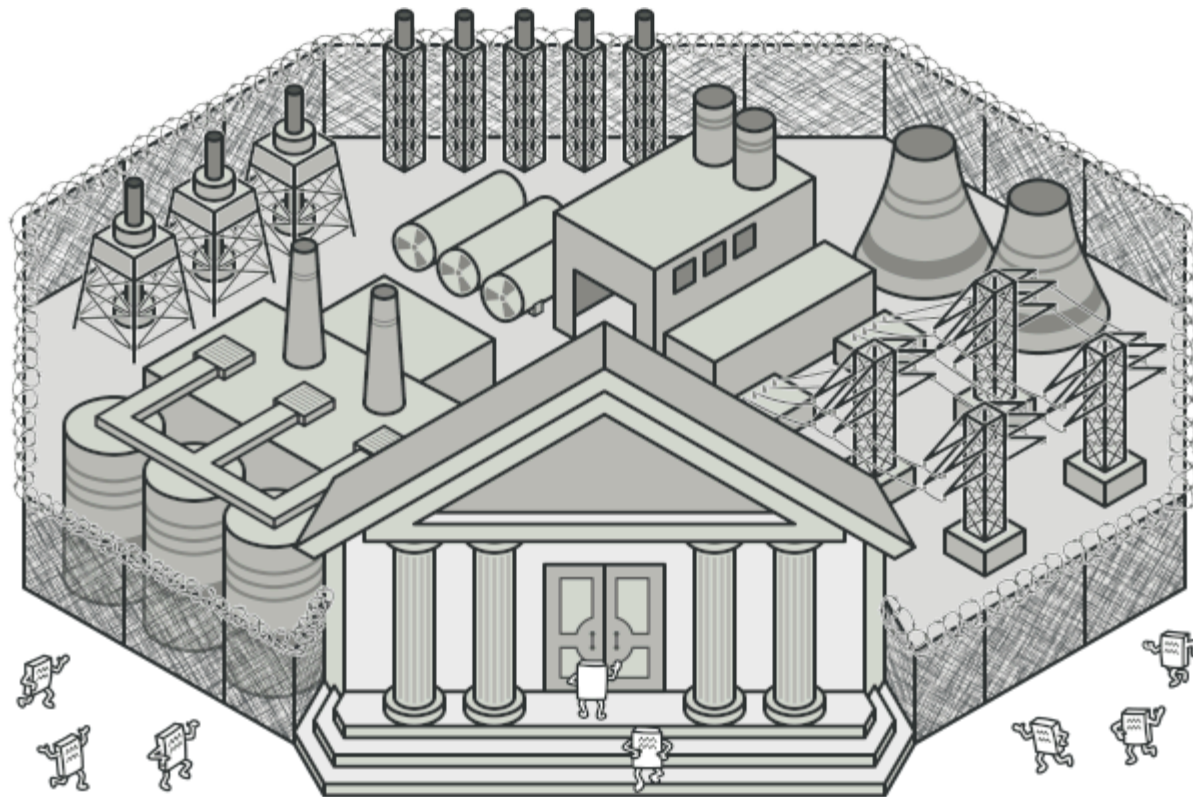
Адаптер объектов:

- позволяет одному адаптеру Adapter **работать со многим адаптируемыми объектами Adaptee**, то есть с самим Adaptee и его подклассами (если таковые имеются). Адаптер может добавить новую функциональность сразу всем адаптируемым объектам;
- **затрудняет замещение операций класса Adaptee**. Для этого потребуется породить от Adaptee подкласс и заставить Adapter ссылаться на этот подкласс, а не на сам Adaptee.



Университет
Сириус
Колледж

Паттерн Фасад (Facade)

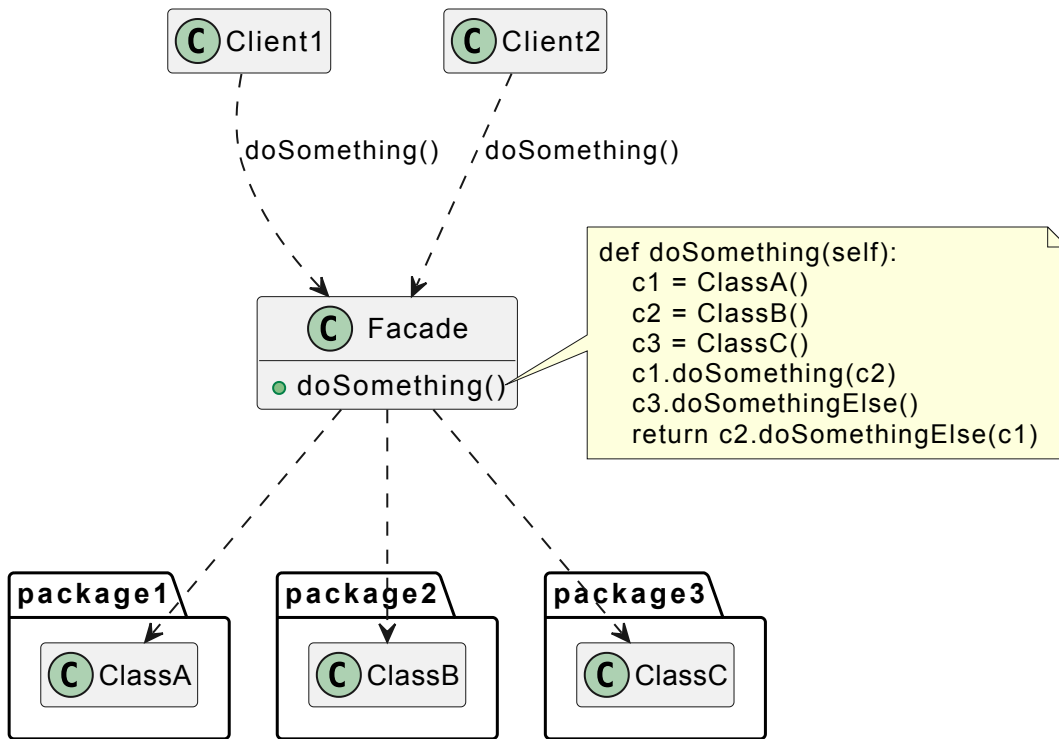


Фасад - паттерн, структурирующий объекты.

Предоставляет унифицированный интерфейс вместо набора интерфейсов некоторой подсистемы. Фасад определяет интерфейс более высокого уровня, который упрощает использование подсистемы.



Паттерн Фасад (Facade)





Паттерн Фасад (Facade)

```
1 class CPU:
2     def freeze(self): print("CPU freezed")
3     def jump(self, position): print(f"CPU jump to {position}")
4     def execute(self): print("CPU execute")
5
6 class Memory:
7     def load(self, position, data): print(f"Memory load {data} to {position}")
8
9 class ComputerFacade:
10     def __init__(self):
11         self.cpu = CPU()
12         self.memory = Memory()
13
14     def start(self):
15         self.cpu.freeze()
```



Паттерн Фасад (Facade)

```
1 class CPU:
2     def freeze(self): print("CPU freezed")
3     def jump(self, position): print(f"CPU jump to {position}")
4     def execute(self): print("CPU execute")
5
6 class Memory:
7     def load(self, position, data): print(f"Memory load {data} to {position}")
8
9 class ComputerFacade:
10     def __init__(self):
11         self.cpu = CPU()
12         self.memory = Memory()
13
14     def start(self):
15         self.cpu.freeze()
```



Паттерн Фасад (Facade)

```
1 class CPU:
2     def freeze(self): print("CPU freezed")
3     def jump(self, position): print(f"CPU jump to {position}")
4     def execute(self): print("CPU execute")
5
6 class Memory:
7     def load(self, position, data): print(f"Memory load {data} to {position}")
8
9 class ComputerFacade:
10     def __init__(self):
11         self.cpu = CPU()
12         self.memory = Memory()
13
14     def start(self):
15         self.cpu.freeze()
```



Паттерн Фасад (Facade)

```
7         def load(self, position, data): print(f"Memory load {data} to {position}")
8
9     class ComputerFacade:
10         def __init__(self):
11             self.cpu = CPU()
12             self.memory = Memory()
13
14         def start(self):
15             self.cpu.freeze()
16             self.memory.load(0, "BOOT_LOADER")
17             self.cpu.jump(0)
18             self.cpu.execute()
19
20 # Клиенту нужна только одна кнопка
21 ComputerFacade().start()
```



Паттерн Фасад (Facade)

```
4     def execute(self): print("CPU execute")
5
6     class Memory:
7         def load(self, position, data): print(f"Memory load {data} to {position}")
8
9     class ComputerFacade:
10         def __init__(self):
11             self.cpu = CPU()
12             self.memory = Memory()
13
14         def start(self):
15             self.cpu.freeze()
16             self.memory.load(0, "BOOT_LOADER")
17             self.cpu.jump(0)
18             self.cpu.execute()
```



Паттерн Фасад (Facade)

```
7         def load(self, position, data): print(f"memory load {data} to {position}")
8
9     class ComputerFacade:
10         def __init__(self):
11             self.cpu = CPU()
12             self.memory = Memory()
13
14         def start(self):
15             self.cpu.freeze()
16             self.memory.load(0, "BOOT_LOADER")
17             self.cpu.jump(0)
18             self.cpu.execute()
19
20 # Клиенту нужна только одна кнопка
21 ComputerFacade().start()
```




Паттерн Фасад (Facade)

```
7         def load(self, position, data): print(f"memory load {data} to {position}")
8
9     class ComputerFacade:
10         def __init__(self):
11             self.cpu = CPU()
12             self.memory = Memory()
13
14         def start(self):
15             self.cpu.freeze()
16             self.memory.load(0, "BOOT_LOADER")
17             self.cpu.jump(0)
18             self.cpu.execute()
19
20 # Клиенту нужна только одна кнопка
21 ComputerFacade().start()
```

Используйте паттерн фасад, когда вы:

- хотите предоставить простой интерфейс к сложной подсистеме. Часто подсистемы усложняются по мере развития. Применение большинства паттернов приводит к появлению меньших классов, но в большем количестве. Такую подсистему проще повторно использовать и настраивать под конкретные нужды, но вместе с тем применять подсистему без настройки становится труднее. Фасад предлагает некоторый вид системы по умолчанию, устраивающий большинство клиентов. И лишь те объекты, которым нужны более широкие возможности настройки, могут обратиться напрямую к тому, что находится за фасадом;

Используйте паттерн фасад, когда вы:

- **хотите предоставить простой интерфейс к сложной подсистеме.** Часто подсистемы усложняются по мере развития. Применение большинства паттернов приводит к появлению меньших классов, но в большем количестве. Такую подсистему проще повторно использовать и настраивать под конкретные нужды, но вместе с тем применять подсистему без настройки становится труднее. Фасад предлагает некоторый вид системы по умолчанию, устраивающий большинство клиентов. И лишь те объекты, которым нужны более широкие возможности настройки, могут обратиться напрямую к тому, что находится за фасадом;

Паттерн Фасад (Facade)

- между клиентами и классами реализации абстракции существует много зависимостей. Фасад позволит отделить подсистему как от клиентов, так и от других подсистем, что, в свою очередь, способствует повышению степени независимости и переносимости;
- вы хотите разложить подсистему на отдельные слои. Используйте фасад для определения точки входа на каждый уровень подсистемы. Если подсистемы зависят друг от друга, то зависимость можно упростить, разрешив подсистемам обмениваться информацией только через фасады.

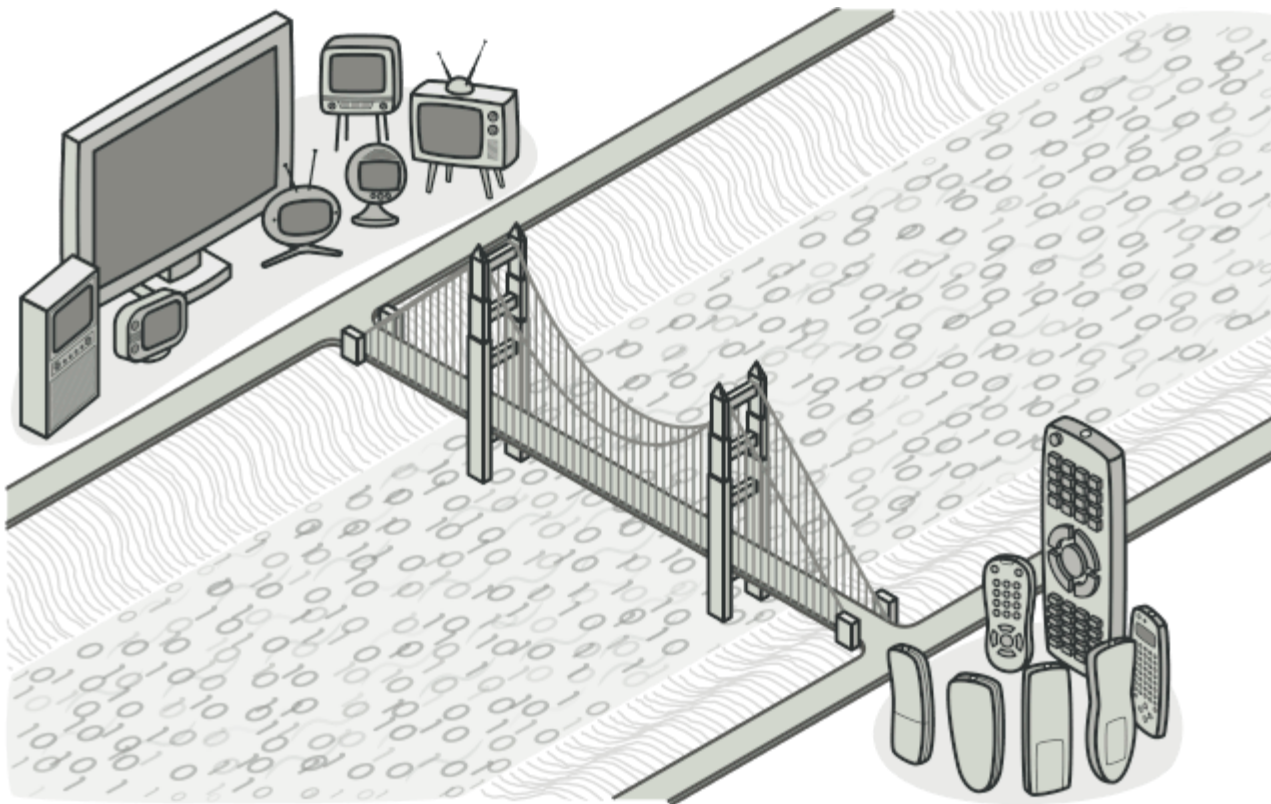
Паттерн Фасад (Facade)

- между клиентами и классами реализации абстракции существует много зависимостей. Фасад позволит отделить подсистему как от клиентов, так и от других подсистем, что, в свою очередь, способствует повышению степени независимости и переносимости;
- вы хотите разложить подсистему на отдельные слои. Используйте фасад для определения точки входа на каждый уровень подсистемы. Если подсистемы зависят друг от друга, то зависимость можно упростить, разрешив подсистемам обмениваться информацией только через фасады.



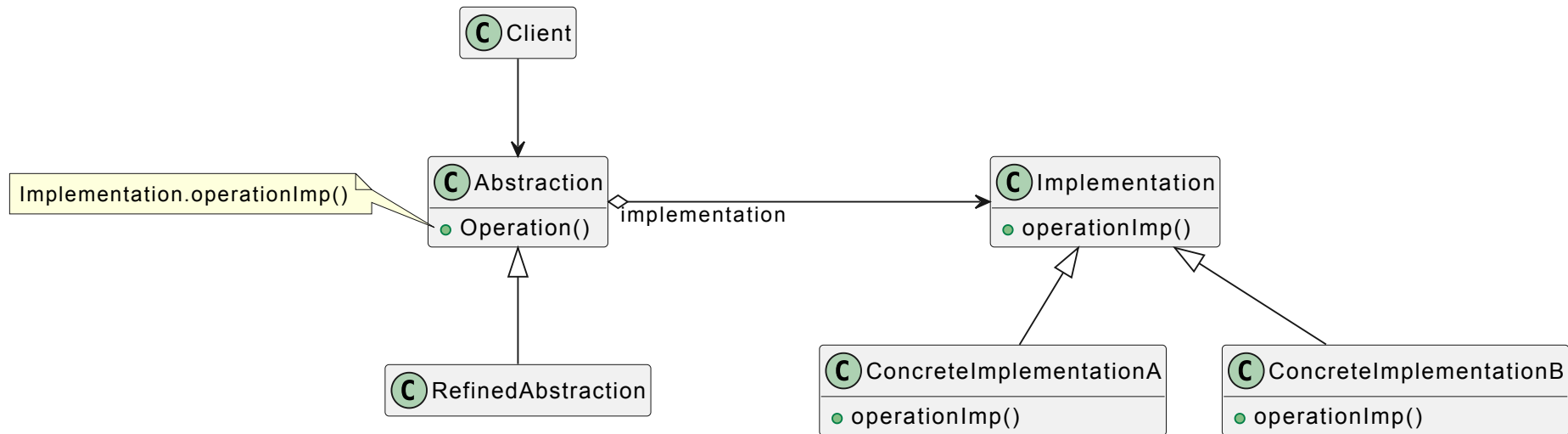
Паттерн Фасад (Facade)

- между клиентами и классами реализации абстракции существует много зависимостей. Фасад позволит отделить подсистему как от клиентов, так и от других подсистем, что, в свою очередь, способствует повышению степени независимости и переносимости;
- вы хотите разложить подсистему на отдельные слои. Используйте фасад для определения точки входа на каждый уровень подсистемы. Если подсистемы зависят друг от друга, то зависимость можно упростить, разрешив подсистемам обмениваться информацией только через фасады.



Мост - паттерн, структурирующий объекты.

Отделить абстракцию от ее реализации так, чтобы то и другое можно было изменять независимо.



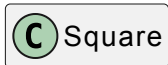


Университет
Сириус
Колледж

Паттерн Мост (Bridge)



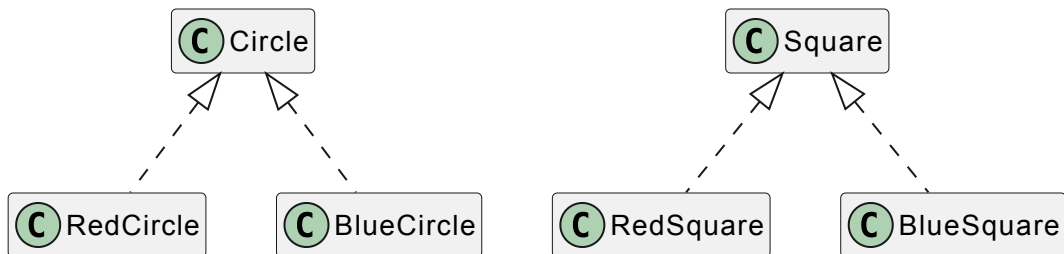
Circle



Square

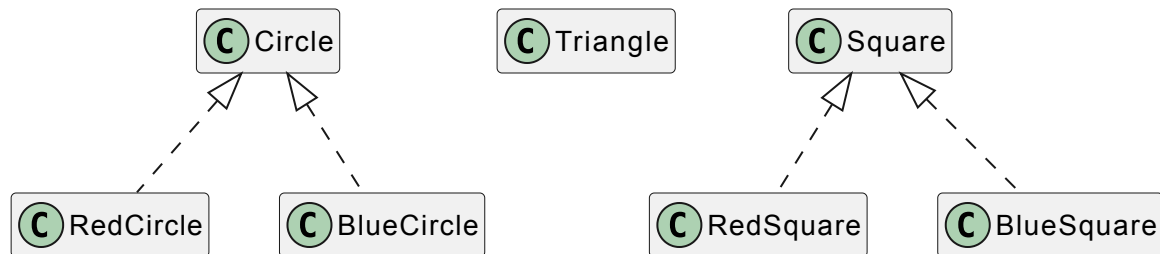


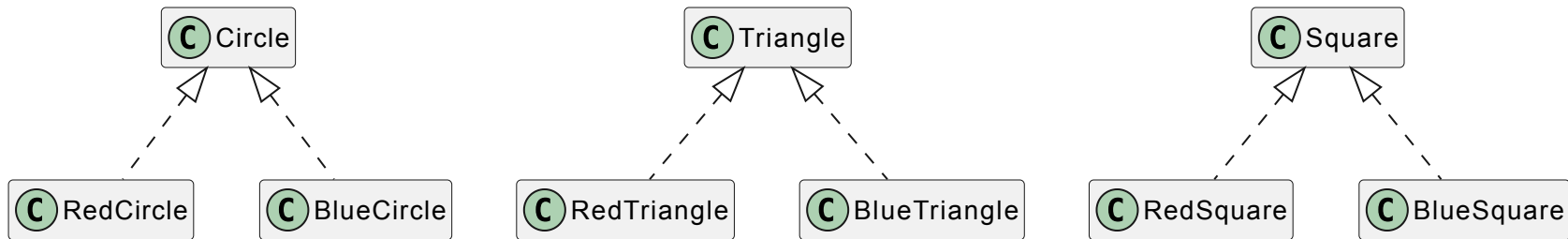
Паттерн Мост (Bridge)

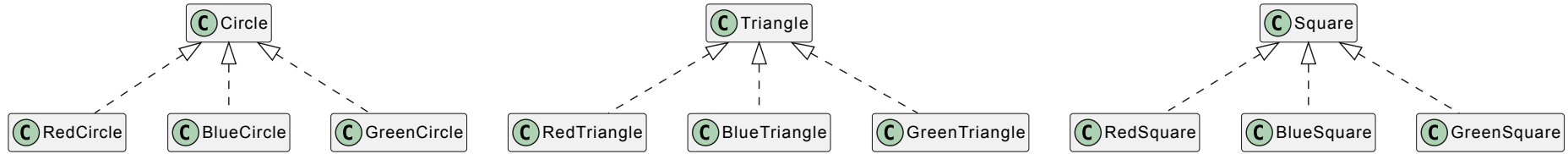




Паттерн Мост (Bridge)





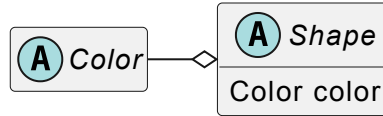


Количество классов растет как $N \times M$ (декартово произведение). Это «**Ад наследования**».



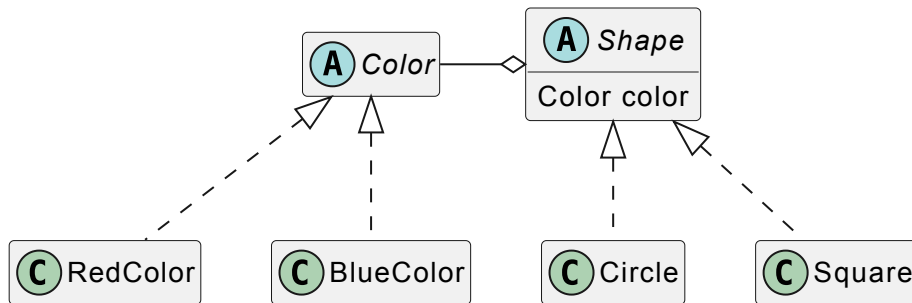
Университет
Сириус
Колледж

Паттерн Мост (Bridge)





Паттерн Мост (Bridge)





```
1  from abc import ABC, abstractmethod
2
3  # === 1. Иерархия Реализации (Цвета) ===
4  class Color(ABC):
5      @abstractmethod
6      def fill(self):
7          pass
8
9  class RedColor(Color):
10     def fill(self):
11         return "Красный"
12
13 class BlueColor(Color):
14     def fill(self):
15         return "Гиний"
```



2

3 # === 1. Иерархия Реализации (Цвета) ===

4 class Color(ABC):

5 @abstractmethod

6 def fill(self):

7 pass

8

9 class RedColor(Color):

10 def fill(self):

11 return "Красный"

12

13 class BlueColor(Color):

14 def fill(self):

15 return "Синий"

16



```
14     def fill(self):
15         return "Синий"
16
17 # === 2. Иерархия Абстракции (Фигуры) ===
18 class Shape(ABC):
19     def __init__(self, color: Color):
20         self.color = color # <--- МОСТ (Bridge)
21
22     @abstractmethod
23     def draw(self):
24         pass
25
26 class Circle(Shape):
27     def draw(self):
28         print(f"Рисуем Круг. Цвет: {self.color.fill()}")
```



```
13 class BlueColor(Color):
14     def fill(self):
15         return "Синий"
16
17 # === 2. Иерархия Абстракции (Фигуры) ===
18 class Shape(ABC):
19     def __init__(self, color: Color):
20         self.color = color # <--- МОСТ (Bridge)
21
22     @abstractmethod
23     def draw(self):
24         pass
25
26 class Circle(Shape):
27     def draw(self):
```



Паттерн Мост (Bridge)

```
23     def draw(self):
24         pass
25
26 class Circle(Shape):
27     def draw(self):
28         print(f"Рисуем Круг. Цвет: {self.color.fill()}")
29
30 class Square(Shape):
31     def draw(self):
32         print(f"Рисуем Квадрат. Цвет: {self.color.fill()}")
33
34 # === 3. Клиентский код ===
35 red = RedColor()
36 blue = BlueColor()
37
```



```
28         print(f"Рисуем круг. Цвет: {self.color.fill()}")
29
30 class Square(Shape):
31     def draw(self):
32         print(f"Рисуем Квадрат. Цвет: {self.color.fill()}")
33
34 # === 3. Клиентский код ===
35 red = RedColor()
36 blue = BlueColor()
37
38 circle_red = Circle(red)
39 square_blue = Square(blue)
40
41 circle_red.draw()
42 square_blue.draw()
```

Используйте паттерн мост, когда:

- хотите избежать постоянной привязки абстракции к реализации. Так, например, бывает, когда реализацию необходимо выбирать во время выполнения программы;
- и абстракции, и реализации должны расширяться новыми подклассами. В таком случае паттерн мост позволяет комбинировать разные абстракции и реализации и изменять их независимо;
- изменения в реализации абстракции не должны сказываться на клиентах, то есть клиентский код не должен перекомпилироваться;
- число классов начинает быстро расти. Это признак того, что иерархию следует разделить на две части;
- вы хотите разделить одну реализацию между несколькими объектами (быть может, применяя подсчет ссылок), и этот факт необходимо скрыть от клиента.

Используйте паттерн мост, когда:

- хотите избежать постоянной привязки абстракции к реализации. Так, например, бывает, когда реализацию необходимо выбирать во время выполнения программы;
- и абстракции, и реализации должны расширяться новыми подклассами. В таком случае паттерн мост позволяет комбинировать разные абстракции и реализации и изменять их независимо;
- изменения в реализации абстракции не должны сказываться на клиентах, то есть клиентский код не должен перекомпилироваться;
- число классов начинает быстро расти. Это признак того, что иерархию следует разделить на две части;
- вы хотите разделить одну реализацию между несколькими объектами (быть может, применяя подсчет ссылок), и этот факт необходимо скрыть от клиента.

Используйте паттерн мост, когда:

- хотите избежать постоянной привязки абстракции к реализации. Так, например, бывает, когда реализацию необходимо выбирать во время выполнения программы;
- и абстракции, и реализации должны расширяться новыми подклассами. В таком случае паттерн мост позволяет комбинировать разные абстракции и реализации и изменять их независимо;
- изменения в реализации абстракции не должны сказываться на клиентах, то есть клиентский код не должен перекомпилироваться;
- число классов начинает быстро расти. Это признак того, что иерархию следует разделить на две части;
- вы хотите разделить одну реализацию между несколькими объектами (быть может, применяя подсчет ссылок), и этот факт необходимо скрыть от клиента.

Используйте паттерн мост, когда:

- хотите избежать постоянной привязки абстракции к реализации. Так, например, бывает, когда реализацию необходимо выбирать во время выполнения программы;
- и абстракции, и реализации должны расширяться новыми подклассами. В таком случае паттерн мост позволяет комбинировать разные абстракции и реализации и изменять их независимо;
- изменения в реализации абстракции не должны сказываться на клиентах, то есть клиентский код не должен перекомпилироваться;
- число классов начинает быстро расти. Это признак того, что иерархию следует разделить на две части;
- вы хотите разделить одну реализацию между несколькими объектами (быть может, применяя подсчет ссылок), и этот факт необходимо скрыть от клиента.

Используйте паттерн мост, когда:

- хотите избежать постоянной привязки абстракции к реализации. Так, например, бывает, когда реализацию необходимо выбирать во время выполнения программы;
- и абстракции, и реализации должны расширяться новыми подклассами. В таком случае паттерн мост позволяет комбинировать разные абстракции и реализации и изменять их независимо;
- изменения в реализации абстракции не должны сказываться на клиентах, то есть клиентский код не должен перекомпилироваться;
- число классов начинает быстро расти. Это признак того, что иерархию следует разделить на две части;
- вы хотите разделить одну реализацию между несколькими объектами (быть может, применяя подсчет ссылок), и этот факт необходимо скрыть от клиента.

Используйте паттерн мост, когда:

- хотите избежать постоянной привязки абстракции к реализации. Так, например, бывает, когда реализацию необходимо выбирать во время выполнения программы;
- и абстракции, и реализации должны расширяться новыми подклассами. В таком случае паттерн мост позволяет комбинировать разные абстракции и реализации и изменять их независимо;
- изменения в реализации абстракции не должны сказываться на клиентах, то есть клиентский код не должен перекомпилироваться;
- число классов начинает быстро расти. Это признак того, что иерархию следует разделить на две части;
- вы хотите разделить одну реализацию между несколькими объектами (быть может, применяя подсчет ссылок), и этот факт необходимо скрыть от клиента.

- отделение реализации от интерфейса. Реализация больше не имеет постоянной привязки к интерфейсу. Реализацию абстракции можно конфигурировать во время выполнения. Объект может даже динамически изменять свою реализацию. Разделение классов `Abstraction` и `Implementor` устраняет также зависимости от реализации, устанавливаемые на этапе компиляции. Чтобы изменить класс реализации, вовсе не обязательно перекомпилировать класс `Abstraction` и его клиентов. Это свойство особенно важно, если необходимо обеспечить двоичную совместимость между разными версиями библиотеки классов.

- **отделение реализации от интерфейса.** Реализация больше не имеет постоянной привязки к интерфейсу. Реализацию абстракции можно конфигурировать во время выполнения. Объект может даже динамически изменять свою реализацию. Разделение классов `Abstraction` и `Implementor` устраняет также зависимости от реализации, устанавливаемые на этапе компиляции. Чтобы изменить класс реализации, вовсе не обязательно перекомпилировать класс `Abstraction` и его клиентов. Это свойство особенно важно, если необходимо обеспечить двоичную совместимость между разными версиями библиотеки классов.

- кроме того, такое разделение облегчает разбиение системы на слои и тем самым позволяет улучшить ее структуру. Высокоуровневые части системы должны знать только о классах `Abstraction` и `Implementor` ;
- повышение степени расширяемости. Можно расширять независимо иерархии классов `Abstraction` и `Implementor` ;
- сокрытие деталей реализации от клиентов. Клиентов можно изолировать от таких деталей реализации, как разделение объектов класса `Implementor` и сопутствующего механизма подсчета ссылок.

- кроме того, такое разделение **облегчает разбиение системы на слои** и тем самым позволяет улучшить ее структуру. Высокоуровневые части системы должны знать только о классах `Abstraction` и `Implementor` ;
- повышение степени расширяемости. Можно расширять независимо иерархии классов `Abstraction` и `Implementor` ;
- сокрытие деталей реализации от клиентов. Клиентов можно изолировать от таких деталей реализации, как разделение объектов класса `Implementor` и сопутствующего механизма подсчета ссылок.

- кроме того, такое разделение **облегчает разбиение системы на слои** и тем самым позволяет улучшить ее структуру. Высокоуровневые части системы должны знать только о классах `Abstraction` и `Implementor` ;
- **повышение степени расширяемости**. Можно расширять независимо иерархии классов `Abstraction` и `Implementor` ;
- сокрытие деталей реализации от клиентов. Клиентов можно изолировать от таких деталей реализации, как разделение объектов класса `Implementor` и сопутствующего механизма подсчета ссылок.

- кроме того, такое разделение **облегчает разбиение системы на слои** и тем самым позволяет улучшить ее структуру. Высокоуровневые части системы должны знать только о классах `Abstraction` и `Implementor` ;
- **повышение степени расширяемости** Можно расширять независимо иерархии классов `Abstraction` и `Implementor` ;
- **сокрытие деталей реализации от клиентов**. Клиентов можно изолировать от таких деталей реализации, как разделение объектов класса `Implementor` и сопутствующего механизма подсчета ссылок.

У паттернов адаптер и мост есть несколько общих атрибутов. Тот и другой повышают гибкость, вводя дополнительный уровень косвенности при обращении к другому объекту. Оба перенаправляют запросы другому объекту, используя иной интерфейс.

Основное различие между адаптером и мостом в их назначении.

- Цель адаптера - устранить несовместимость между двумя существующими интерфейсами. При разработке адаптера не учитывается, как эти интерфейсы реализованы и то, как они могут независимо развиваться в будущем. Он должен лишь обеспечить совместную работу двух независимо разработанных классов, так чтобы ни один из них не пришлось переделывать.
- С другой стороны, мост связывает абстракцию с ее, возможно, многочисленными реализациями. Данный паттерн предоставляет клиентам стабильный интерфейс, позволяя в то же время изменять классы, которые его реализуют.

У паттернов адаптер и мост есть несколько общих атрибутов. Тот и другой повышают гибкость, вводя дополнительный уровень косвенности при обращении к другому объекту. Оба перенаправляют запросы другому объекту, используя иной интерфейс.

Основное различие между адаптером и мостом в их назначении.

- **Цель адаптера - устранить несовместимость между двумя существующими интерфейсами.** При разработке адаптера не учитывается, как эти интерфейсы реализованы и то, как они могут независимо развиваться в будущем. Он должен лишь обеспечить совместную работу двух независимо разработанных классов, так чтобы ни один из них не пришлось переделывать.
- С другой стороны, мост связывает абстракцию с ее, возможно, многочисленными реализациями. Данный паттерн предоставляет клиентам стабильный интерфейс, позволяя в то же время изменять классы, которые его реализуют.

У паттернов адаптер и мост есть несколько общих атрибутов. Тот и другой повышают гибкость, вводя дополнительный уровень косвенности при обращении к другому объекту. Оба перенаправляют запросы другому объекту, используя иной интерфейс.

Основное различие между адаптером и мостом в их назначении.

- Цель адаптера - устранить несовместимость между двумя существующими интерфейсами. При разработке адаптера не учитывается, как эти интерфейсы реализованы и то, как они могут независимо развиваться в будущем. Он должен лишь обеспечить совместную работу двух независимо разработанных классов, так чтобы ни один из них не пришлось переделывать.
- С другой стороны, мост связывает абстракцию с ее, возможно, многочисленными реализациями. Данный паттерн предоставляет клиентам стабильный интерфейс, позволяя в то же время изменять классы, которые его реализуют.

Когда выясняется, что два несовместимых класса должны работать вместе, следует обратиться к адаптеру. Тем самым удастся избежать дублирования кода.

Наоборот, пользователь моста с самого начала понимает, что у абстракции может быть несколько реализаций и развитие того и другого будет идти независимо

Адаптер обеспечивает работу после того, как нечто спроектировано; мост - до того.

Фасад можно представлять себе как адаптер к набору других объектов. Но при такой интерпретации легко не заметить такой нюанс: фасад определяет новый интерфейс, тогда как адаптер повторно использует уже имеющийся. Адаптер заставляет работать вместе два существующих интерфейса, а не определяет новый.

Когда выясняется, что два несовместимых класса должны работать вместе, следует обратиться к адаптеру. Тем самым удастся избежать дублирования кода.

Наоборот, пользователь моста с самого начала понимает, что у абстракции может быть несколько реализаций и развитие того и другого будет идти независимо

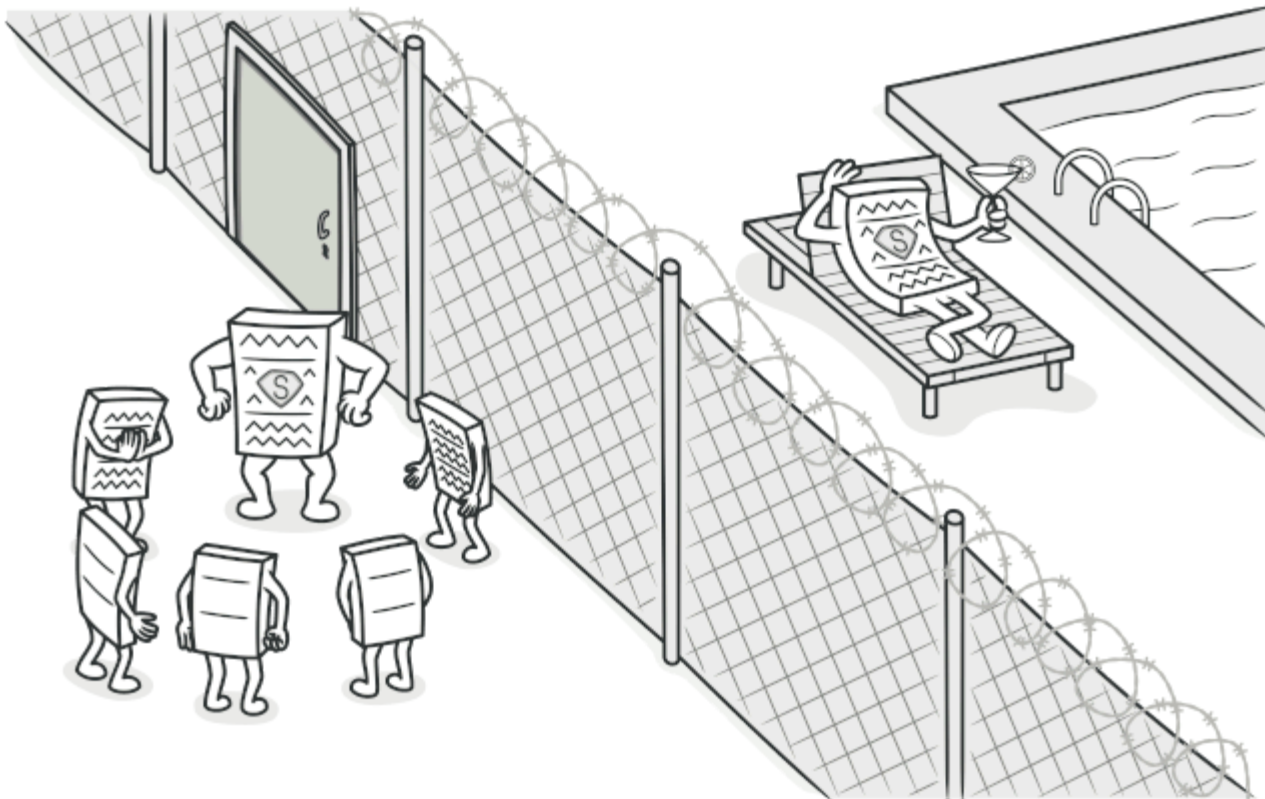
Адаптер обеспечивает работу после того, как нечто спроектировано; мост - до того.

Фасад можно представлять себе как адаптер к набору других объектов. Но при такой интерпретации легко не заметить такой нюанс: **фасад определяет новый интерфейс, тогда как адаптер повторно использует уже имеющийся**. Адаптер заставляет работать вместе два существующих интерфейса, а не определяет новый.



Университет
Сириус
Колледж

Паттерн Заместитель (Proxy)



Заместитель - паттерн, структурирующий объекты.

Является суррогатом другого объекта и контролирует доступ к нему.

Паттерн заместитель применим во всех случаях, когда возникает необходимость сослаться на объект более изощренно, чем это возможно, если использовать простой указатель.

Примеры:

Заместитель - паттерн, структурирующий объекты.

Является суррогатом другого объекта и контролирует доступ к нему.

Паттерн заместитель применим во всех случаях, когда возникает необходимость сослаться на объект более изощренно, чем это возможно, если использовать простой указатель.

Примеры:

- удаленный заместитель может предоставлять доступ к объекту в другом адресном пространстве;

Заместитель - паттерн, структурирующий объекты.

Является суррогатом другого объекта и контролирует доступ к нему.

Паттерн заместитель применим во всех случаях, когда возникает необходимость сослаться на объект более изощренно, чем это возможно, если использовать простой указатель.

Примеры:

- удаленный заместитель может предоставлять доступ к объекту в другом адресном пространстве;
- защищающий заместитель может ограничивать доступ к другому объекту;

Заместитель - паттерн, структурирующий объекты.

Является суррогатом другого объекта и контролирует доступ к нему.

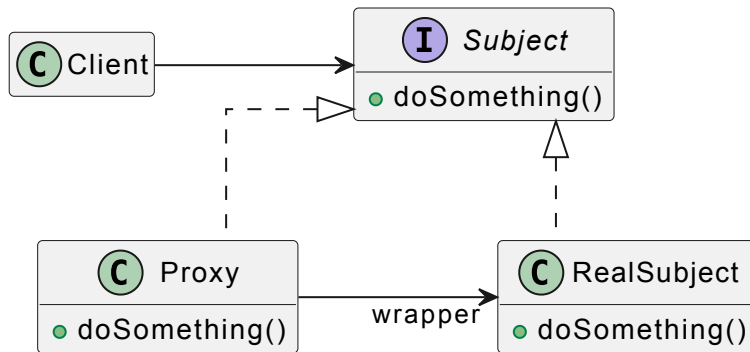
Паттерн заместитель применим во всех случаях, когда возникает необходимость сослаться на объект более изощренно, чем это возможно, если использовать простой указатель.

Примеры:

- удаленный заместитель может предоставлять доступ к объекту в другом адресном пространстве;
- защищающий заместитель может ограничивать доступ к другому объекту;
- виртуальный заместитель может создавать тяжёлые объекты по требованию (изображения, файлы).



Паттерн Заместитель (Proxy)





Паттерн Заместитель (Proxy)

```
1 class RealImage:
2     def __init__(self, filename):
3         self.filename = filename
4         self.load_from_disk()
5
6     def load_from_disk(self):
7         print(f>Loading {self.filename}...")
8
9     def display(self):
10        print(f>Displaying {self.filename}")
11
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
```



Паттерн Заместитель (Proxy)

```
1 class RealImage:
2     def __init__(self, filename):
3         self.filename = filename
4         self.load_from_disk()
5
6     def load_from_disk(self):
7         print(f>Loading {self.filename}...")
8
9     def display(self):
10        print(f>Displaying {self.filename}")
11
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
```



Паттерн Заместитель (Proxy)

```
1  class RealImage:
2      def __init__(self, filename):
3          self.filename = filename
4          self.load_from_disk()
5
6      def load_from_disk(self):
7          print(f>Loading {self.filename}...")
8
9      def display(self):
10         print(f>Displaying {self.filename}")
11
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
```




Паттерн Заместитель (Proxy)

```
9     def display(self):
10         print(f"Displaying {self.filename}")
11
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
16
17     def display(self):
18         if self.real_image is None:
19             self.real_image = RealImage(self.filename) # Ленивая загрузка
20             self.real_image.display()
21
22 # Картинка не грузится при создании прокси
23 img = ProxyImage("photo.jpg")
```



Паттерн Заместитель (Proxy)

```
7         print(f>Loading {self.filename}...")
8
9     def display(self):
10         print(f>Displaying {self.filename}")
11
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
16
17     def display(self):
18         if self.real_image is None:
19             self.real_image = RealImage(self.filename) # Ленивая загрузка
20             self.real_image.display()
21
```



Паттерн Заместитель (Proxy)

```
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
16
17     def display(self):
18         if self.real_image is None:
19             self.real_image = RealImage(self.filename) # Ленивая загрузка
20             self.real_image.display()
21
22 # Картинка не грузится при создании прокси
23 img = ProxyImage("photo.jpg")
24 print("Прокси создан")
25 # Картинка грузится только сейчас
26 img.display()
```



Паттерн Заместитель (Proxy)

```
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
16
17     def display(self):
18         if self.real_image is None:
19             self.real_image = RealImage(self.filename) # Ленивая загрузка
20             self.real_image.display()
21
22 # Картинка не грузится при создании прокси
23 img = ProxyImage("photo.jpg")
24 print("Прокси создан")
25 # Картинка грузится только сейчас
26 img.display()
```



Паттерн Заместитель (Proxy)

```
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
16
17     def display(self):
18         if self.real_image is None:
19             self.real_image = RealImage(self.filename) # Ленивая загрузка
20         self.real_image.display()
21
22 # Картинка не грузится при создании прокси
23 img = ProxyImage("photo.jpg")
24 print("Прокси создан")
25 # Картинка грузится только сейчас
26 img.display()
```



Паттерн Заместитель (Proxy)

```
12 class ProxyImage:
13     def __init__(self, filename):
14         self.filename = filename
15         self.real_image = None
16
17     def display(self):
18         if self.real_image is None:
19             self.real_image = RealImage(self.filename) # Ленивая загрузка
20             self.real_image.display()
21
22 # Картинка не грузится при создании прокси
23 img = ProxyImage("photo.jpg")
24 print("Прокси создан")
25 # Картинка грузится только сейчас
26 img.display()
```

- удаленный заместитель может скрыть тот факт, что объект находится в другом адресном пространстве;
- виртуальный заместитель может выполнять оптимизацию, например создание объекта по требованию;
- защищающий заместитель и «умная» ссылка позволяют решать дополнительные задачи при доступе к объекту.